

Opportunities Platform — Project Plan (v1)

1. Project Description

A web platform that aggregates educational and career opportunities — internships, master's programs, exchange programs, scholarships, and similar postings — in one searchable place.

Admins/Content Managers curate the content: they create, edit, publish, unpublish, and delete opportunity posts using a structured template.

Regular Users/Visitors browse and search the published opportunities. They have read-only access — no accounts required unless you decide otherwise (see clarifying questions below).

The core value proposition is *organization and discoverability*: instead of opportunities being scattered across university sites, mailing lists, and social media, they live in one place with consistent structure and good filtering.

2. User Roles and Permissions

Action	Admin/Content Manager	Regular User/Visitor
View published opportunities	✓	✓
Search / filter opportunities	✓	✓
View opportunity detail page	✓	✓
Create a post	✓	✗
Edit a post	✓	✗
Publish / unpublish a post	✓	✗
Delete a post	✓	✗
View unpublished/draft posts	✓	✗

Action	Admin/Content Manager	Regular User/Visitor
Manage other admin accounts	Depends — see clarifying questions	✗

This maps cleanly onto **Spring Security roles**: `ROLE_ADMIN` and `ROLE_USER` (or no role at all for anonymous visitors, if you don't require visitor accounts).

3. Main Website Pages

- Home / Landing Page** — highlights featured or recently added opportunities, quick category shortcuts, and a search bar.
- Browse/Listing Page** (per category or all) — the main "catalog" view with filters and a paginated list of opportunity cards.
- Opportunity Detail Page** — full information about one opportunity.
- Search Results Page** — could be the same component as the Browse page, driven by query parameters.
- Admin Dashboard** — list of all posts (including drafts/expired), with create/edit/delete controls.
- Create/Edit Opportunity Form** (admin only).
- Login Page** (admin authentication at minimum).
- 404 / Not Found and Error pages.**

4. Main Features per Page

Home Page

- Featured or newest opportunities
- Category navigation (Internships, Scholarships, Master's Programs, Exchange Programs, Other)
- Global search bar

Browse/Listing Page

- Filter sidebar or bar: opportunity type, country, city, university/organization, scholarship availability (yes/no), deadline range
- Sort options (e.g., soonest deadline first, newest posted)
- Pagination or infinite scroll
- Each item shown as a compact "card": title, type, country/city, deadline, funding badge

Opportunity Detail Page

All structured fields (see section 5)
Clear "days left" or "expired" indicator
Official application link (outbound)
Possibly a "similar opportunities" section later

Admin Dashboard

Table of all posts with status (draft/published/expired)
Quick actions: edit, publish/unpublish, delete
Filter by status/type for management purposes

Create/Edit Form

Structured input matching the required fields
Validation (e.g., deadline must be a future date to publish, required fields enforced)

5. Required Information for Every Opportunity Post

Based on what you listed, plus a few practical additions:

Title
Opportunity type (Internship / Master's / Exchange / Scholarship / Other)
Description
Country
City
University or organization
Eligibility requirements
Application instructions
Application deadline
Scholarship or funding information
Official application link

Status (Draft / Published / Archived) — needed to control visibility

Created date / last updated date — useful for sorting and auditing

Created by (which admin) — useful for accountability once you have multiple admins

6. Suggested Domain Models / Entities

User — id, name, email, password (hashed), role (ADMIN/USER)

Opportunity — id, title, description, type, country, city, organizationName, eligibility, applicationInstructions, applicationDeadline, fundingInfo, applicationLink, status, createdAt, updatedAt, createdBy

OpportunityType — could start as a Java `enum` (INTERNSHIP, MASTERS, EXCHANGE, SCHOLARSHIP, OTHER) rather than a full entity — simpler, and you don't need admins creating new types dynamically unless you want that flexibility later.

Entities you *could* add later but don't need for v1:

Organization (if you want organizations to have their own profile pages with logos, descriptions, multiple opportunities linked)

Tag (for finer-grained topic filtering beyond type/country/city)

Bookmark/SavedOpportunity (only relevant if regular users have accounts)

7. Relationships Between Entities

`User (ADMIN)` **1 → many** `Opportunity` (one admin creates many posts; each post has one creator)

`Opportunity` **many → 1** `OpportunityType` (if you make it an entity instead of an enum)

Country/City/University are simple string fields for v1 — no separate entities needed unless you want normalized, autocomplete-friendly filtering later (e.g., a `Country` lookup table). Starting with strings is simpler and easier to build; you can normalize later once you see real data.

8. Recommended Implementation Order

Project skeleton — Spring Boot project setup (Spring Web, Spring Data JPA, Validation, Spring Security added but not configured yet), package structure (controller/service/repository/model).

Domain model — define the `Opportunity` entity and `OpportunityType` enum, no database yet — just get the Java classes right and validated with unit tests or simple console checks.

In-memory or H2 database — wire up Spring Data JPA with H2 so you can persist and query without setting up a real database yet.

Repository layer — Spring Data JPA repository with basic CRUD, then custom query methods for filtering (by type, country, etc.).

Service layer — business logic: e.g., automatically treating opportunities as "expired" when `applicationDeadline` has passed, filtering out unpublished posts from public queries.

Public REST endpoints — list/search/filter opportunities, get one opportunity by id. Test with Postman/curl — no frontend needed yet.

Admin REST endpoints — create/edit/delete/publish. Still unsecured at this point, just to get logic working.

Authentication & authorization — add Spring Security, lock down admin endpoints, leave public endpoints open.

Validation & error handling — proper request validation, consistent error responses.

Switch to a real database (PostgreSQL/MySQL) once the schema feels stable.

Frontend — once the API is solid and documented, build the UI against it.

This order lets you validate your domain logic and API design early, without frontend or security complexity slowing you down.

9. Things You May Have Overlooked (and Questions for You)

Do regular users need accounts at all? Right now your spec says they only search/filter/view — that can be fully anonymous, no login required. If you want bookmarking/saved searches/deadline alerts later, you'd need visitor accounts. Worth deciding now since it affects the User entity design.

Expired handling: you said "clearly marked or automatically hidden" — these are two different behaviors. Do you want both (e.g., hidden from default browse view, but still viewable if directly linked, with an "expired" badge)?

Draft vs Published workflow: should new posts go live immediately, or should there be a draft state before publishing? Your spec mentions "publish" as a separate admin action, which implies a draft state — worth confirming.

Multiple admins: will there be one admin account, or many? If many, do all admins have equal permissions, or do you want tiers (e.g., a super-admin who can manage other admin accounts)?

Duplicate/similar postings: with many opportunities from many sources, do you want any duplicate-detection or is that a manual admin concern?

Time zones for deadlines: application deadlines often have specific time zones — worth deciding whether to store deadline as date-only or full datetime with time zone.

Search: simple filter-based search (dropdowns) is easy to implement first; full-text search (searching within title/description) is a nice-to-have that can come later.